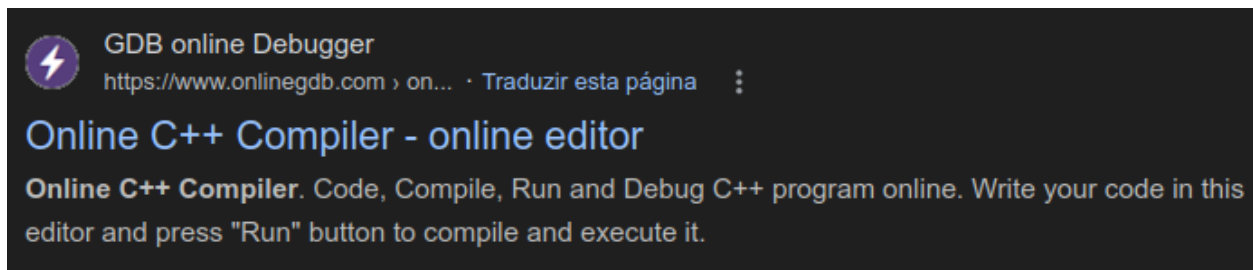


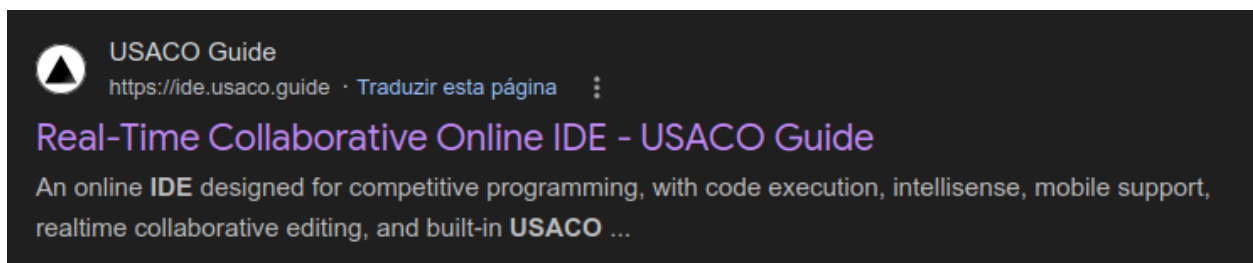
Aula 1 - Primeiros Passos em C++

Início

Nesse início de curso, nós vamos trabalhar apenas com editores de texto e compiladores online, isto é, vamos programar em sites. Os dois mais recomendados para começar são o **GDB** e o **usaco**.



The screenshot shows the homepage of the GDB online Debugger. It features a purple lightning bolt icon in a circle. The text includes 'GDB online Debugger', the URL 'https://www.onlinegdb.com', a link to 'Traduzir esta página', and the title 'Online C++ Compiler - online editor'. A description below states: 'Online C++ Compiler. Code, Compile, Run and Debug C++ program online. Write your code in this editor and press "Run" button to compile and execute it.'



The screenshot shows the homepage of the USACO Guide. It features a black triangle icon in a white circle. The text includes 'USACO Guide', the URL 'https://ide.usaco.guide', a link to 'Traduzir esta página', and the title 'Real-Time Collaborative Online IDE - USACO Guide'. A description below states: 'An online IDE designed for competitive programming, with code execution, intellisense, mobile support, realtime collaborative editing, and built-in USACO ...'

O GDB é bem intuitivo. No USACO, cliquem em "create new file" e depois apenas "new file". Em ambos haverá um botão para rodar o código. Enquanto no GDB vocês precisam fornecer as entradas via terminal em tempo real de execução, no USACO vocês fornecem todas as entradas previamente, antes de executar, e depois apenas analisam a saída.

Fundamentos

Saídas

```
#include <iostream>
using namespace std;

int main(){
    cout << "Olá, mundo!" << endl;
}
```

Essa é a estrutura básica de um programa em C++. Vocês não precisam **ainda** saber o que aquelas duas primeiras linhas significam, por enquanto, apenas aceitem que elas são necessárias para que os códigos de vocês funcionem.

Tudo que estiver contido dentro do **main** será executado de cima para baixo assim que vocês clicarem em "run code".

O `cout` é a função padrão de **saída** do C++, ele é usado para imprimir informações na tela. O `endl` é uma partícula especial que imprime uma **quebra de linha**, isto é, pula uma linha. Ele não é estritamente necessário, o `cout` funciona sem ele sem problemas.

Variáveis

As variáveis em programação são espaços de memória reservados no seu computador com o intuito de armazenar informações de diferentes tipos. Em C++, nós precisamos seguir o procedimento padrão de **declarar variáveis** antes de usá-las, isso significa dar um nome a uma variável e dizer seu tipo:

```
int x; // uma variável inteira, armazena números inteiros positivos e negativos
float y; // ponto flutuante, armazena números reais
char c; // caractere, armazena letras e números na forma de caractere singular
bool b; // verdadeiro ou falso
string s; // cadeia de caracteres, texto
```

```
<tipo da variável> <nome da variável>;
```

Uma vez declarada uma variável, nós podemos atribuir valores a elas:

```
x = 2;  
y = 4.0931;  
c = 'G'; // sempre ao armazenar caracteres, usem aspas SIMPLE  
S  
b = true;  
s = "Matias"; // e em cadeias, usem aspas DUPLAS
```

Vale ressaltar que a partícula `=` não significa "é igual a". Na programação, ele é um operador de **atribuição**. `x = 2` pode ser lido como "x recebe 2" ou "x agora vale 2".

Entradas

Uma vez que conseguimos salvar informações e mostrar informações pro usuário, só nos falta **pedir informações do usuário**. Isso é feito com o `cin`, a função padrão de entrada do C++;

```
int x;  
cin >> x;
```

Ele funciona com o operador oposto ao do `cout`, o `>>`. No código acima, nós declaramos a existência da variável `x`, sem valor, e depois perguntamos ao usuário qual o valor de `x`.

Depois de salvar o valor de `x`, podemos brincar com ele mostrando-o de diferentes formas:

```
cout << x;  
cout << x+2;  
cout << x*3;  
cout << x*x;
```

Se x fosse 2, por exemplo, teríamos 2, 4, 6 e 4 novamente como saída para esse programa.

Operadores aritméticos, booleanos e comparativos

Operadores aritméticos são aqueles que funcionam entre valores e variáveis numéricas:

```
z = x + y; // soma
z = x - y; // subtração
z = x * y; // multiplicação
z = x / y; // divisão
z = x % y; // resto da divisão

// essas duas barras aqui indicam um comentário
// tudo que estiver depois delas não será lido pelo computador
// na hora da execução
// elas servem pra explicar trechos de códigos
```

Enquanto operadores booleanos funcionam entre valores e variáveis lógicas:

```
z = a || b; // OU lógico
z = a && b; // E lógico
z = !a; // NÃO lógico
```

E operadores comparativos são usados em condicionais para fazer comparações entre números:

```
a == b; // verifica se A é igual a B
a != b; // verifica se A é diferente de B
a > b;
a < b;
a >= b; // maior ou igual
a <= b;
```

Vale ressaltar que, via de regra, os operadores seguem a ordem de precedência da matemática comum (divisão vem antes de adição, por exemplo), mas, parênteses podem ser usados, da mesma maneira que na matemática.

Condicionais

Até o momento, todos os nossos códigos seguem um fluxo muito linear de instruções. Vamos quebrar isso agora. As condicionais servem para criar ramos de execução que podem ou não ser executados a depender de certas condições:

```
if(condição){  
    // faça isso se a condição é verdade;  
}  
else if(condição2){  
    // se a última condição não era verdade e a condição2 é v  
erdade, faça isso;  
}  
else{  
    // se NADA deu certo, faça isso;  
}
```

Vocês podem adicionar quantos `else if` vocês quiserem, mas apenas um `if` e um `else` podem existir dentro de uma cadeia condicional dessas.

Estruturas de Repetição

Em alguns momentos, nós queremos repetir um conjunto de linhas de código várias vezes, digitá-las é inviável normalmente, então, usamos estruturas de repetição. Existem 3 delas no total.

For

```
for(inicio; condicao; fim){  
    // repita isso  
}
```

`inicio` é uma linha de código que será executada **ANTES** do loop começar,
`condicao` é uma condição que será checada **A CADA** iteração do loop, se ela for falsa, o loop acaba

`fim` é uma linha de código que será executada **DEPOIS DE CADA** iteração do loop.

Tudo que estiver dentro das chaves será executado um número de vezes.

Exemplo de `for` na prática:

```
for(int i = 0; i<10; i++){  
    cout << i << endl;  
}
```

Esse loop roda dez vezes, imprimindo o valor de `i` a cada iteração. `i++` é um açúcar sintático que significa `i = i + 1`, basicamente, aumenta o valor de `i` em uma unidade. O `i--` existe também!

While

```
while(condicao){  
    // repita isso  
}
```

Bem intuitivo, “enquanto a condicao for verdadeira, repita isso”. É possível simular o `for` de antes com um `while`:

```
int i = 0;  
while(i < 10){  
    cout << i << endl;  
    i++;  
}
```

Do While

```
do{  
    // repita isso  
}while(condicao);
```

Extremamente menos usado, é a mesma coisa do `while` normal, mas ele testa a condição **DEPOIS** de cada iteração.

Vocês já devem ter percebido a esse ponto, mas é importante deixar claro, depois de cada instrução individual em C++, **coloquem um ponto e vírgula**, pelo amor de Deus.